Journal of Building Engineering 103 (2025) 112189

Contents lists available at ScienceDirect

Journal of Building Engineering

journal homepage: www.elsevier.com/locate/jobe

Improving knowledge management in building engineering with hybrid retrieval-augmented generation framework

Zhiqi Wang [a], Zhongcun Liu [b], Weizhen Lu [b], Lu Jia [a,*]

[a] School of Infrastructure Engineering, Nanchang Univ., 999# Xuefu Ave., Honggutan District, Nanchang, Jiangxi, 330000, China
[b] ZhongMei Engineering Group Ltd., Kowloon Lake International Expo Center, Shangrao St., Honggutan District, Nanchang, Jiangxi, 330000, China

ARTICLE INFO

Keywords:
Retrieval augmented generation
Large language model
Knowledge engineering
Risk and error recognition
Reasoning

ABSTRACT

Domain knowledge is a critical asset that provides the foundation for a range of building activities, including but not limited to design, construction, operation, and maintenance. However, existing approaches to knowledge engineering often require extensive up-front work and are limited in generalizability. In recent years, large language models (LLMs) have undergone significant growth and demonstrated notable generalization and reasoning capabilities. Nevertheless, the deployment of LLMs in the field of buildings has been constrained by several factors, including hallucinations, a lack of domain-specific knowledge, difficulties in updating knowledge, and data security concerns. This study aims to develop an improved retrieval augmented generation (RAG) framework based on domain characteristics for general, intelligent, and scalable AI-based applications. The primary advancement is the proposal of a hybrid index based on vectors, a property graph, and keywords, accompanied by a corresponding hybrid retrieval strategy. To evaluate the proposed framework comprehensively, three tasks were designed and executed: basic queries, risk and error recognition in complex queries, and open text generation. The results show that the improved RAG significantly outperforms the baseline and the native LLM in all three tasks. This study advances the integration of domain knowledge with generative AI and demonstrates the potential of LLM-RAG-based solutions for application in building lifecycle management.

1. Introduction

The building-related industry has a lengthy history and a broad scope. Over the course of its development, a substantial corpus of knowledge has accumulated, including legal and regulatory frameworks, corporate standards, project management protocols, and more. In a typical workflow, practitioners need to plan, judge, make decisions, and take action on the basis of knowledge. Effective knowledge management has been shown to be an important way to support decision-making and to enable organizations to gain a competitive advantage [1–3]. Knowledge engineering (KE), represented by semantic ontologies and knowledge graphs (KGs), has been shown to be an effective approach for facilitating knowledge management and application [4]. A typical KE system relies on a knowledge base, a rule set, and a logical structure. These components enable the KE system to produce clear and interpretable results through reliable reasoning. Nevertheless, the main limitation of KE is its lack of generalizability, which presents a challenge in addressing problems that are vague, uncertain, or situated beyond the boundaries of predefined logic [5]. For example, the Semantic

* Corresponding author.
E-mail address: jialuuser@163.com (L. Jia).

https://doi.org/10.1016/j.jobe.2025.112189
Received 7 November 2024; Received in revised form 15 February 2025; Accepted 19 February 2025
Available online 21 February 2025
2352-7102/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Web is an effective tool for automated reasoning on a specific issue. However, when the issue expands, it can be time-consuming to compile rules for each potential scenario [6]. The KG has the potential to serve as a large-scale knowledge base. However, it is constrained by challenges such as long-tail relationships, which would make it too bulky and difficult to perform complex and fine-grained automated reasoning tasks [7,8]. When deployed as a customized reasoning engine, the challenge persists in storing the complete knowledge context.

The large language model (LLM) has potential, to a certain extent, in the form of generalized intelligence. When encountering problems in real life, in addition to resorting to search engines, an increasing number of users are turning to LLMs as a source of assistance. LLMs can even answer some specialized questions. They seem to be intelligent and knowledgeable experts. Indeed, LLMs do possess notable reasoning and generative capabilities [9–11], yet the knowledge they express is essentially a series of probability-based outcomes [12,13]. In other words, LLMs predict knowledge rather than accurately knowing it, which renders them less reliable when confronted with the challenge of addressing problems in specialized domains directly. Furthermore, LLMs are prone to output information that is outdated, unfaithful, or contains hallucinations [14,15], especially in knowledge-intensive tasks and specialized domains [16]. This can have a detrimental effect on the accuracy and robustness of any automated program.

Practitioners require more intelligent and dependable methods to effectively access knowledge and facilitate decision-making in their workflows. If we can provide LLMs with the requisite knowledge while posing queries, we may be able to develop a general-purpose, intelligent, and scalable AI assistant, which is the concept of retrieval augmented generation (RAG). The present RAG framework is mainly composed of two parts: the plug-in knowledge base and the LLM. In a typical usage scenario, a RAG system retrieves relevant knowledge from the knowledge base based on a user query. Subsequently, the system provides the retrieved knowledge along with the query to the LLM to obtain an enhanced response. However, existing RAG methods are mainly applied to the common domain and lack optimization for specialized domains. This leads to limitations in the structuring of specialized knowledge bases, and further leads to insufficient knowledge retrieval accuracy and knowledge utilization efficiency. In terms of application performance, the main limitation of existing methods is insufficient response correctness, which is particularly pronounced in complex query and reasoning tasks. In light of the foregoing considerations, this study aims to enhance the retrieval performance of the RAG system for knowledge in the domain of buildings through optimization based on domain characteristics, with the ultimate objective of providing more reasonable answers to user queries in representative usage scenarios of the building lifecycle. To this end, this study proposes a RAG framework that has been enhanced for the characterization of domain knowledge. Inspired by KGs, we present a new hybrid index structure. This structure employs a combination of vectors, a property graph, and keywords. It maintains the integrity and coherence of knowledge while allowing vector-based semantic retrieval, thus providing a robust and flexible foundation for the RAG framework. A corresponding node parsing method is also provided to automate the processing from document to index. In terms of the retrieval strategy, we improve the retrieval process, as well as the query transformation and node post-processing modules, to accommodate the potential query patterns that may arise in actual projects. To evaluate the proposed framework comprehensively, three tasks were designed: basic queries, risk and error recognition in complex queries, and open text generation. The results demonstrate that the performance of the improved RAG markedly surpasses that of the baseline and the native LLM in each task.

The remainder of the paper is organized as follows. Section 2 provides an overview of knowledge engineering in the field of buildings, as well as an examination of RAG research and applications. Section 3 presents the proposed RAG framework. Section 4 describes the evaluation configurations and tasks. Section 5 presents the evaluation results and discussion. Section 6 concludes the study.

2. Related works

2.1. Knowledge management and engineering in buildings

Knowledge management is a long-standing topic in the field of buildings [17,18]. Early KE research focused primarily on the construction of expert knowledge bases [19], knowledge acquisition, and knowledge mining [20]. As theory has developed, an increasing number of studies have proposed the use of semantic ontology and KG-based approaches for the representation and reasoning of domain knowledge. Jiang et al. [21] used a KG to visually describe and identify unsafe hoisting operations for early warning. This approach achieved over 70 % accuracy, but the fine-grained KG used was limited in scale and lacked an automated construction method. Rasmussen et al. [22] developed a KG that conforms to property management ontology standards to manage interrelated project information. This approach allows dynamic assessment of design changes and captures design logic beyond geometric information. Zhong et al. [6] employed an ontological approach to facilitate the representation and reasoning of construction plans, thereby ensuring compliance with construction technology programs. However, this study also noted that the development of ontologies can be time-consuming and requires much preliminary work. While these methods are effective in representing and reasoning about specific problems, they exhibit constraints with respect to automation, generalization, and scalability.

To overcome these limitations, some studies have begun to explore the combination of KE with deep neural networks (DNNs). Wu et al. [23] developed a named entity recognition (NER) model based on a combined algorithm to automate the extraction of entities and relationships in construction accident cases. This approach contributes to automating the construction of KGs. Pan et al. [24] proposed a framework for the extraction of entities via computer vision to extend the knowledge sources that are supported by KGs. Related works have concentrated on the extraction of entities and relationships, largely because previous management and reasoning models have relied on the abstraction of knowledge. The completeness of knowledge and the generalizability of reasoning models have not been sufficiently addressed.

2.2. Retrieval augmented generation

LLM has become an important foundation for a range of tasks in the field of NLP [25]. However, the limitations of LLM in terms of hallucinations, lack of training data for specialized knowledge, high cost of knowledge updates, and data security concerns have posed significant challenges to its application in the professional domain. RAG offers an optimal solution to these concerns [26]. The typical implementation of RAG is based on an LLM and an external knowledge base (often called an index) [27,28]. The knowledge documents are initially split into easily manipulable chunks, which are then embedded and stored in an index based on a vector database. Upon entering a query, the RAG first retrieves the chunks most similar to the query in the index and then provides the retrieval results to the LLM for an augmented response. Providing knowledge in context can improve response performance and reduce the occurrence of hallucinations compared to direct responses [29]. In addition, the use of a customized knowledge base eliminates the need for costly domain-specific training and improves data security [30]. The RAG paradigm is implemented and optimized in a modular fashion. Table 1 provides a summary of the typical RAG modules and associated methods.

2.3. RAG in specialized fields

Compared to other industries, the field of medical informatics has paid more attention to RAG. Several recent studies have demonstrated the potential of RAG to aid diagnosis [47,48], process medical records [49,50], and improve healthcare services [51,52]. Chandrasekhar et al. [53] introduced RAG into the field of additive manufacturing and demonstrated that the RAG system has better domain knowledge and specificity than does GPT-4. Siddharth et al. [54] used RAG to structure engineering design facts from patent text based on existing engineering knowledge. While the RAG systems used in these studies may not have undergone substantial modular improvements, their performance consistently surpassed that of the native LLMs in all instances. Wong et al. [55] sought to develop an LLM application with the capacity for human-like logical reasoning to identify potential risks in construction contracts. To this end, they proposed and evaluated a customized RAG method. The results indicate that this method is capable of stable performance in real-world scenarios. These studies show that RAG exhibits robust generality and scalability. Additionally, the domain-specific optimization of the RAG approach can lead to further improvements in retrieval and response performance.

3. Methodology

This study proposes an improved RAG framework based on domain characteristics. The framework consists of three main parts: hybrid index, hybrid retrieval strategy, and response synthesis. The overall workflow is shown in Fig. 1.

3.1. Hybrid index

An index is a specific data structure designed to store knowledge in accordance with a certain pattern and to support retrieval. The structure of an index is typically determined by the characteristics of the original document and can significantly influence the choice of retrieval strategy. Existing RAG systems generally employ vector indexes. This indexing structure usually splits knowledge documents into text chunks of a fixed length, followed by the embedding of these text chunks to obtain dense semantic vectors. These vectors are then stored as records in a vector database and matched with a specific vector similarity metric. Vector indexes are widely used in common domains because of their high generality. However, in the context of engineering documents related to buildings, the utilization of such vector records has the potential to compromise the completeness and relevance of the knowledge, consequently leading to a deterioration in the retrieval performance for specialized queries. While RAG has made notable advancements in the general field, its research and applications in specialized fields, particularly in the domain of buildings, remain constrained. It is

Table 1
Typical RAG modules and associated methods.

Modules	Descriptions	Methods
Parser	Parse original knowledge documents into the smallest unit needed for indexing	Chunking with fixed length Chunking by sentence Semantic chunking [31]
Index	A data structure for managing knowledge and supporting retrieval	Text embedding [32–34] Vector database [29] Graph database [35]
Query transformer	Used to transform the original query into a more easily retrievable form	Query rewriting [36] Query decomposition [37] Pseudo-document generation [38,39]
Retriever	Return the knowledge most similar to the query according to a certain pattern	Approximate nearest neighbor [40] Keyword filter
Postprocessor	Further processing of retrieved results to improve response performance	Similarity-based re-ranking [41] Word frequency-based re-ranking [42,43] LLM-based re-ranking
Response synthesizer	Output response based on query and retrieved results	Repacking [44] Retrieval fusion [45,46]

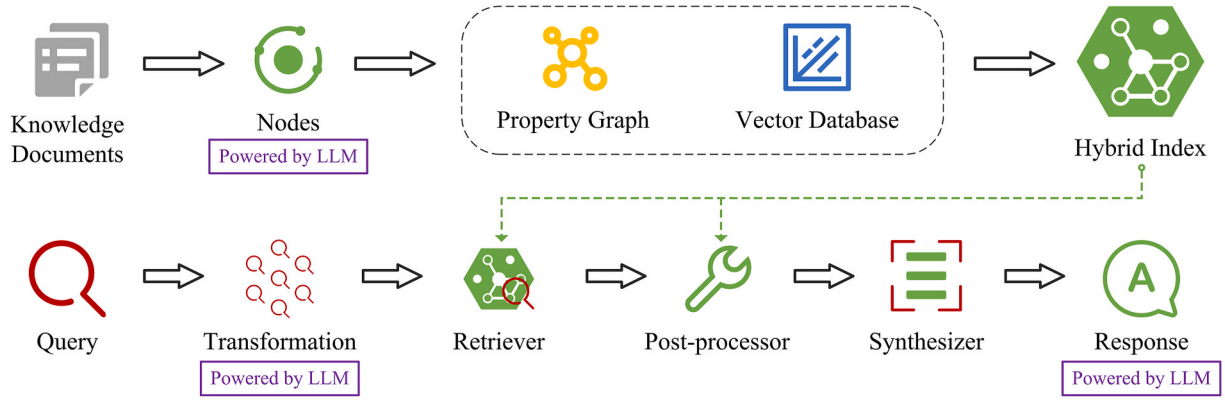

Fig. 1. Proposed RAG workflow.

therefore essential to design a more suitable index structure. In general, engineering documents in the field of buildings exhibit the following characteristics when compared to conventional documents:

1) Engineering documents are not as unstructured as prose but rather exhibit a degree of explicit structure, which may be described as semi-structured.
2) Frequently, there are references or complementary relationships between paragraphs.
3) A response to a query may require knowledge from different sources.
4) A substantial number of tables and illustrations are included.

Considering the above factors, this study presents a hybrid index structure, as illustrated in Fig. 2. This structure better maintains the completeness and relevance of knowledge while supporting flexible vector matching and structure-based knowledge complementation, thereby further improving retrieval performance.

The hybrid index uses nodes, similar to text chunks in a vector index, as the basic unit. These nodes are classified into five categories, as illustrated in Table 2. The primary function of nodes is to serve as repositories of knowledge and to participate in the retrieval

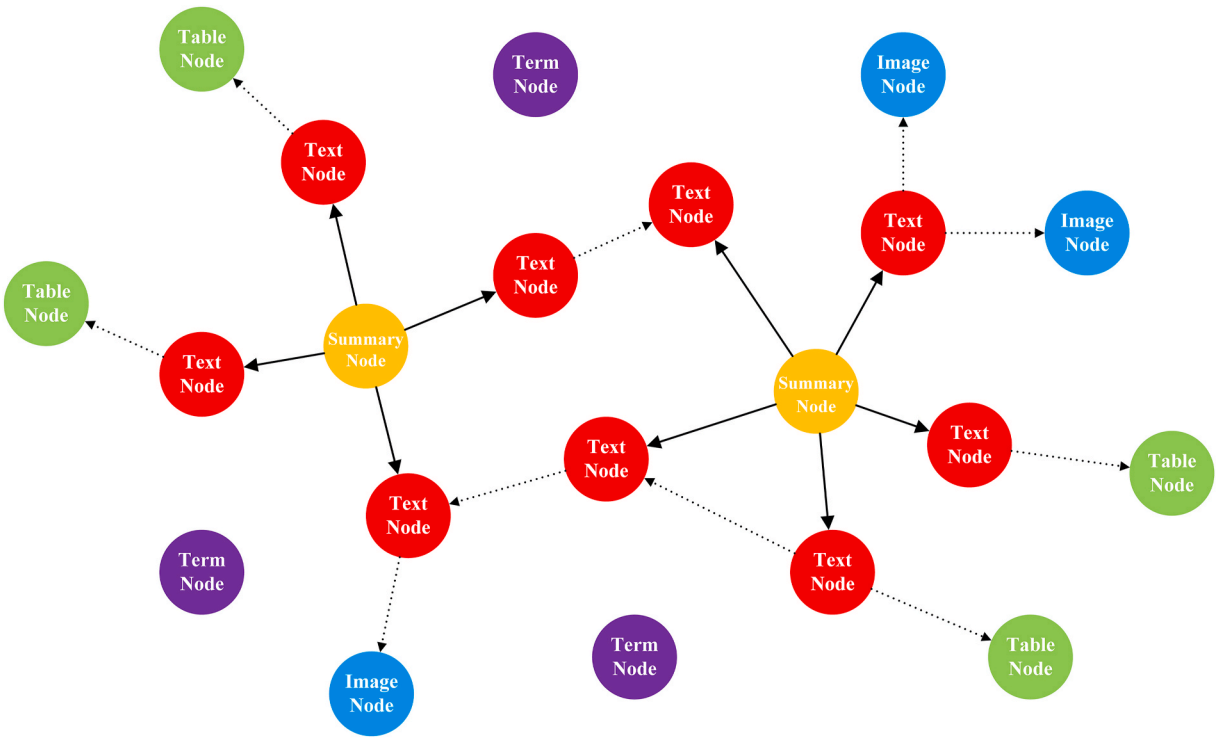

Fig. 2. Hybrid index structure.

process. They are connected to each other through References and Contains relationships. The embedding vectors are stored in a dedicated vector database, while relationships and other properties are stored as a property graph.

The TextNode represents the most crucial node type within the hybrid index, as it serves to convey the vast majority of the knowledge contained within the index. The ID is the unique identifier of a node. The recommended ID pattern is a combination of the document number, clause number, and semantic number. The Text and Keywords properties serve to record the text and keywords of the knowledge. The embedded text is stored in the Embedding property. The IDs of references to other text, tables, and images in the text are recorded in the References property. The properties of the same name in other nodes are set in a similar manner to those above.

In most cases, TermNodes are not connected to other nodes because terms are not typically referenced directly. Their Text property records both terms and definitions, as both types of information may appear in a query.

SummaryNodes do not store knowledge themselves. Instead, they summarize and link the knowledge of several related TextNodes. In some cases, summarized text is more likely to be retrieved. The main reason for this is that a piece of knowledge may not provide a comprehensive account of its context, but rather directly state requirements, methods, or conclusions. This may not only diminish the similarity between the query and target nodes but also result in the loss of the applicability conditions of the knowledge. Summarizing these TextNodes as a whole will facilitate their retrieval and response during a query. This process is relatively straightforward and efficient for semi-structured documents.

TableNodes and ImageNodes are designed to address multimodal concerns. For the reliability of the retrieval program, only their captions can be embedded and participate in the retrieval process. The content of tables may prove an impediment to vector matching, while the images themselves may present difficulties in embedding or accurate matching owing to their specialized and abstract nature. In canonical formats, tables and images are referenced in text. As a result, TableNodes and ImageNodes are typically referenced by TextNodes, which increases the likelihood that they will be retrieved.

3.1.1. Node parsing

Node quality is critical to retrieval results. While it is possible to create nodes manually with high quality, automated parsing methods can markedly enhance efficiency. Vector indexes typically chunk documents by sentence or fixed length. While these methods are highly efficient, they inevitably destroy semantic integrity. In addition, extracting metadata from text has historically been a complex and expensive task, as it usually involves training or fine-tuning NER models in specialized domains [56]. The use of LLM has gradually become more prevalent for a range of NLP tasks as its performance has improved. While LLMs already demonstrate a strong instruction-following capability, placing complete trust in probability-based models is not advisable, as this may compromise the robustness of automated programs.

Given the above considerations, this study employs the function calling response mode and the Pydantic method to automatically extract node properties from the text for TextNodes, TermNodes, and SummaryNodes. Function calling is the response mode used by LLMs when calling external tools. A helper object is created as a tool, the properties to be extracted are set as tool parameters, and the LLM is forced to call the tool. In this mode, the LLM returns the requisite set of parameters for calling the tool, namely, the extracted property values, based on the user's input. Subsequently, the returned data is validated via the Pydantic method to ensure that the fields comply with the construction requirements of the nodes. This method yields satisfactory parsing results and avoids expensive training and fine-tuning. A set of referenceable parameter descriptions and prompt templates for zero-shot learning is presented in Table 3. TableNodes and ImageNodes can be parsed by hardcoding based on their format specificity.

3.1.2. Relationship connection

A relationship is a concept from graph theory. It is represented in a property graph as a directed edge connecting two nodes. Two types of relationships, References and Contains, are designed in the hybrid index. TextNodes and SummaryNodes store the IDs of the nodes to which they are connected in the References and Contains properties, respectively. Iterate over all TextNodes and SummaryNodes, pointing directed edges from the original node to the target node to connect all relationships in the graph. The introduction of relationships allows for the implementation of more flexible and diverse retrieval strategies. Some nodes that are not readily retrievable due to their inherent characteristics can also be enhanced by connected nodes.

3.1.3. Text embedding and similarity matching

The Embedding property of a node is a high-dimensional vector. The upper limit of the dimensions supported by common embedding models is typically between 128 and 3072. The purpose of embedding is to create digital representations of text that can be used in mathematical computations.

Cosine similarity represents a principal approach for measuring the degree of similarity between two non-zero vectors. This is

Table 2
Node labels and basic properties.

Label	TextNode	TermNode	SummaryNode	TableNode	ImageNode
Basic Properties	ID: str Text: str Embedding: [float] Keywords: [str] References: [str]	ID: str Text: str Embedding: [float]	ID: str Text: str Embedding: [float] Keywords: [str] Contains: [str]	ID: str Caption: str Embedding: [float] Content: str Notes: str	ID: str Caption: str Embedding: [float] File_path: str

Table 3
Parameter descriptions and prompt templates for node parsing.

Property	Type	Description
Text (TextNode)	List[str]	Small chunks of text based on semantics
Text (TermtNode)	List[str]	Terms and their definitions specified in the text
Text (SummaryNode)	Str	A concise summary of the text
Keywords	List[str]	Keywords or unique entities in the text
References	List[str]	Names or numbers of the references in the text
Prompt Template	Here is a section of a document in the field of construction engineering: {context} Given the contextual information, extract out a {tool_name} object.	

achieved through the calculation of the vector angle between the query's embedding and the node's embedding at the time of retrieval, as illustrated in Eq. (1). The smaller the angle is, the greater the similarity in direction between the vectors, indicating that the texts are more similar. The advantages of cosine similarity include the following: (1) it is not affected by the length of the vectors; (2) it performs well on high-dimensional data; (3) it is more robust to outliers and noise; and (4) the range of values is bounded, making it easy to interpret and compare. Fig. 3 presents a collection of 40 text embeddings related to engineering, safety, management, and the environment. The figure illustrates the similarity between these embeddings after dimensionality reduction.

$$\textit{Cosine Similarity}\ (\boldsymbol{a}, \boldsymbol{b}) = \frac{\boldsymbol{a} \bullet \boldsymbol{b}}{\|\boldsymbol{a}\|\ \|\boldsymbol{b}\|} = \frac{\sum_{i=1}^{n} a_i b_i}{\sqrt{\sum_{i=1}^{n} (a_i)^2}\sqrt{\sum_{i=1}^{n} (b_i)^2}} \tag{1}$$

where $\boldsymbol{a}$ and $\boldsymbol{b}$ are n-dimensional non-zero vectors, and a_i and b_i are their components in each dimension, $i = 1, 2, \ldots, n$.

3.2. Hybrid retrieval strategy

Retrieval is the process of fetching the target node from the index based on a query. One of the most common retrieval methods is "top-k" semantic retrieval, which retrieves the top-k nodes most similar to the query based on vector similarity. This method is easy to implement and highly applicable. However, when the query is lengthy or encompasses multiple aspects, it is typically challenging for this method to guarantee satisfactory recall. To address this limitation, we propose a hybrid retrieval strategy, the flowchart of which is illustrated in Fig. 4. This method is better equipped to handle complex queries and improve retrieval quality by supplementing or filtering nodes based on keywords.

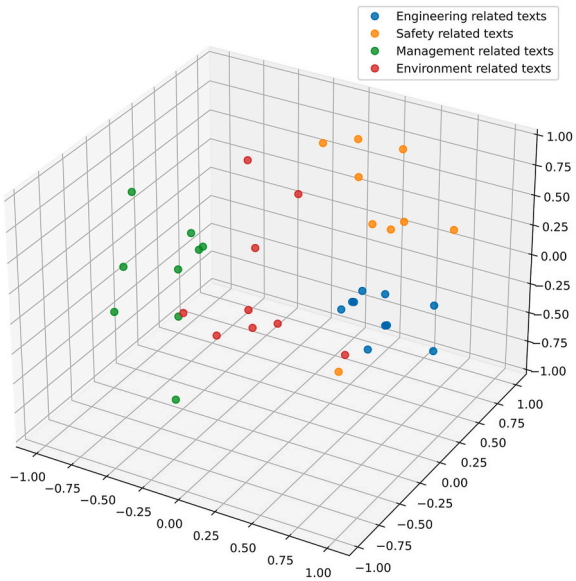

Fig. 3. Dimensionality-reduced visualization of text embedding for different topics.

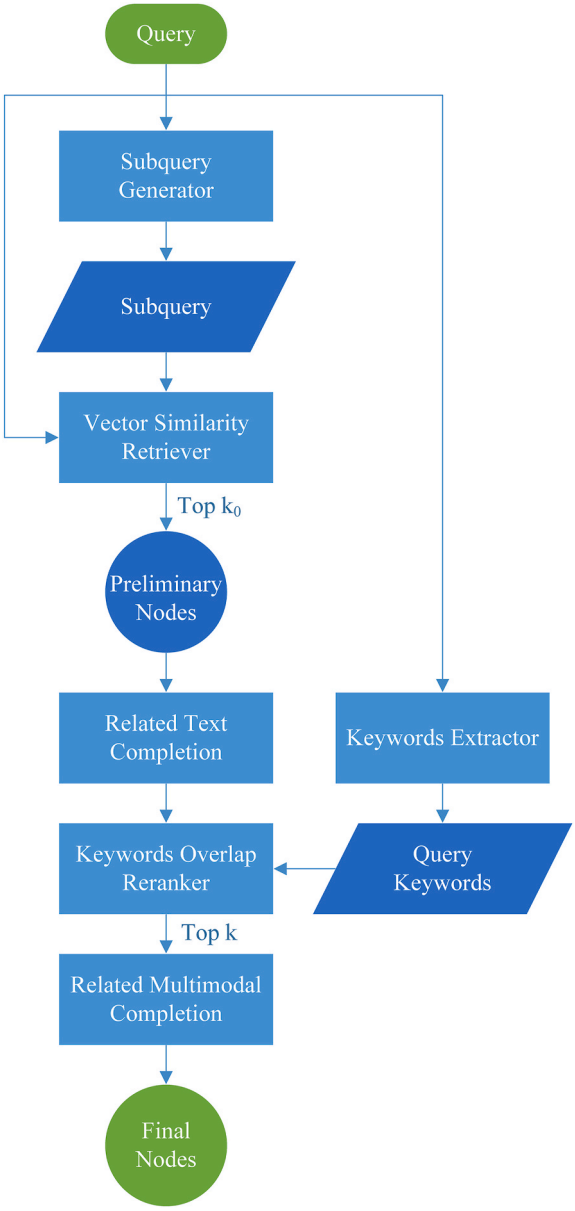

Fig. 4. Workflow of the hybrid retrieval strategy.

3.2.1. Query transformation and retrieval process

When the query is brief and clear, satisfactory results can be obtained by the query vector similarity alone. However, when the query is complex and encompasses multiple topics, these topics can interfere with each other, ultimately resulting in a low similarity between the query and the target node. Consequently, a less relevant node may exhibit greater similarity. In previous research and practice, the subquery method has been proposed to address this problem. This method involves decomposing a complex query into a number of subqueries, which are then retrieved and responded to separately. While subquery generation has the potential to enhance retrieval in certain scenarios, it is not without inherent constraints. First, the conventional subquery method is not concerned with the meaningfulness of the generated subqueries, resulting in the generation of largely inconsequential subqueries. Second, query decomposition inevitably entails the loss of context or conditions present in the original query, which can impede reasoning.

Considering the above factors, an improved prompt template is provided during the query transformation process. Table 4 compares the impacts of the improved template and the conventional template on the subquery generation results. The improved template employs a role-setting and chain-of-thought [10] prompting strategy. This prompts the LLM to determine which information is most likely to be incorrect or risky and then has the subqueries ask the standard requirements for that information. The template allows the user to control the number of subqueries. Reducing the number can enhance the overall quality of the generated subqueries and save

Table 4
Subquery generation.

Item	Improved subquery generation	Conventional subquery generation
Original query	There is a steel bar with the following parameters, please determine whether there is a problem: <Category: Strength: 400Mpa, Diameter: 20 mm, Length: 6000 mm, Cold-drawn Rate: 1.5 %, Bending Diameter: 60 mm, Bending Angle: 75°>	
Prompt Template	You are an expert in civil engineering. Your task is to generate multiple subqueries based on an input query. Please first determine what information is basic or unlikely to be wrong, and what is most likely to be in error or at risk. The subqueries ask what the requirements should be for the information that you think is most likely to be in error or at risk. Generate {num_subqueries} search queries, one on each line, related to the following input query: Query: {original_query} Queries:	You are a helpful assistant that generates multiple search queries based on a single input query. Generate search queries, one on each line, related to the following input query: Query: {original_query} Queries:
Subqueries	1. What are the recommended standards for the cold-drawn rate of a steel bar with a strength of 400Mpa and a diameter of 20 mm? 2. What is the standard bending diameter for steel bars with a 20 mm diameter and a length of 6000 mm? 3. What are the typical bending angle limitations for a 20 mm steel bar with a bending diameter of 60 mm?	1. What is the minimum required strength for steel bars in construction? 2. How does a strength of 400Mpa compare to industry standards for steel bars? 3. What are the typical sizes and diameters for steel bars used in construction? 4. Are there specific guidelines for the length of steel bars in structural applications? 5. How does a cold-drawn rate of 1.5 % affect the mechanical properties of steel bars? 6. What is the acceptable range for cold-drawn rates in steel bar processing? 7. How should the bending diameter of a steel bar be determined? 8. Is a bending diameter of 60 mm within the acceptable range for a 20 mm steel bar? 9. What are standard procedures for bending steel bars to a 75° angle? 10. How does bending angle affect the structural integrity of steel bars? 11. Are there any safety concerns with a bending angle of 75° for a steel bar of this size? 12. What are common issues associated with the parameters provided for this steel bar? 13. How do the dimensional tolerances of steel bars impact their usage in construction?

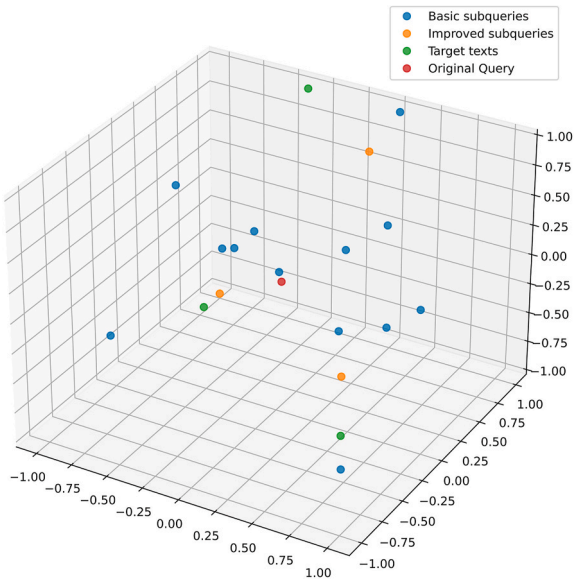

Fig. 5. Dimensionality-reduced visualization of subquery generation.

tokens. When the token length of the LLM is sufficient, relaxing or lifting the limit can further improve the response performance. In this example, the number of subqueries is set to 3. The similarity between the original query, the subqueries, and the target texts is shown in Fig. 5. It can be observed that the original query is farther away from any of the target nodes. A few conventional subqueries may achieve better results, but the improved subqueries show a significant improvement in both similarity and stability.

It is also necessary to implement corresponding improvements in the response process. At the retrieval stage, vector similarity matching is first performed on the original query and subqueries individually, fetching a total of k_0 preliminary nodes. Then, the LLM is no longer required to respond to each subquery separately. Instead, the preliminary nodes will enter the post-processing process and eventually serve to answer the original query.

3.2.2. Node post-processing

The performance of vector similarity-based retrieval can be affected by differences in embedding models, languages, domain knowledge, etc. Keywords are commonly used as supplementary data in RAG pipelines. The advantage of keywords is that they are universally applicable and do not require additional training. In previous research and practice, keywords have typically been used to filter or supplement retrieval results. However, this method does not always work consistently because it is not very controllable. In this study, we have extended the preliminary nodes based on the proposed index structure and devised a re-ranking method based on keyword overlap, with the goal of gently improving the retrieval results. The node post-processing steps are described below:

1) Retrieve the top k_0 nodes (preliminary nodes) with the highest vector similarity.
2) If the preliminary nodes contain SummaryNodes, replace them with connected TextNodes.
3) If the TextNodes in the preliminary nodes reference other TextNodes, include the referenced nodes in the preliminary nodes.
4) Remove duplicate nodes, leaving k_1 nodes.
5) Extract the query keywords.
6) Calculate the correction scores of these nodes separately using the correction formula shown in Eq. (2), Eq. (3), and Eq. (4), where $i = 1, 2, \ldots, k_1$.
7) Take the top k nodes with the highest correction score.
8) If TextNodes reference TableNodes and ImageNodes, include the referenced nodes to obtain the final nodes.

$$s_i = s_{vec,i} + s_{kw,i}\left(s_{vec,max} - s_{vec,i}\right) \tag{2}$$

$$s_{kw,i} = \frac{n_i}{\max\left(n_{kw,q}, n_{kw,i}\right)} \tag{3}$$

$$s_{vec,max} = \max\left(s_{vec,1}, s_{vec,2}, \ldots s_{vec,i}\right) \tag{4}$$

where, for the ith node, s_i is the correction score, $s_{vec,i}$ is the vector similarity score (i.e., cosine similarity), $s_{kw,i}$ is the keyword matching score, n_i is the number of the same keywords in the query and the node, $n_{kw,q}$ is the number of keywords in the query, and $n_{kw,i}$ is the number of keywords in the node.

3.3. Response synthesis

In the response synthesis stage, the knowledge contained in the final nodes is organized into a context and provided to the LLM along with the original query. If the query is complex, the value of k can be set to a larger value. The LLM provides a better response to text near the beginning and end parts when dealing with long contexts [44]. Therefore, when synthesizing contexts, the nodes with higher scores are arranged at the beginning and end, while the nodes with lower scores are situated in the middle. The LLM uses this

Table 5
Base configuration for the example and evaluation tasks.

Method	Proposed framework	Baseline	Native LLM
Documents	Consistent original knowledge documents		–
Input	Query		
Index	Hybrid index	Vector index	–
Retriever	Hybrid retriever	Vector retriever	–
Prompt template (response)	You are an expert in the field of building engineering. Below is the context given. ——————————— {retrieved_nodes} ——————————— Please answer the following questions based on the context above: {query}		You are an expert in the field of building engineering. Please answer the following questions: {query}
LLM	GPT-4O		
Embedding Model	Text-embedding-3-large (1536 dimensions)		
Output	Answer		

knowledge to enhance professionalism and enable knowledge-augmented reasoning and answering.

4. Example and evaluation

In order to validate the effectiveness of the proposed framework, we consider possible application scenarios of RAG in the field of buildings. The construction and execution of the proposed RAG framework are demonstrated with a representative example. Then, three tasks are designed to comprehensively evaluate the proposed framework. Considering the consistency of observations, the evaluation tasks use the same original documents and index as the example. For each evaluation task, we also performed comparison tests using the native LLM and a baseline RAG system (using only a fixed chunk size of 1024 tokens and a cosine similarity retriever). The tests do not perform any few-shot learning or fine-tuning of the LLM. It is important to note that the actual performance of these methods may vary slightly due to the use of different LLMs and embedding models. Considering the performance, recognition, and development ecology of the models, this paper uses GPT-4O and Text-embedding-3-large (1536 dimensions) for testing. The configuration is detailed in Table 5.

4.1. Example

The original engineering knowledge documents need to be collected before building the index. Considering the topic, length, openness, clarity, and avoidance of potential knowledge conflicts, 8 publicly available Chinese building codes were selected as a consistent source of knowledge: GB50010–2010, GB50352–2019, GB50666–2011, GB50870–2013, GB50924–2014, GB55015–2021, GB55016-2021, and GB55031-2022. These documents address common topics in building engineering, including design, construction, materials, environment, energy, safety, and health. In order to implement customized prompt templates and response formats, documents were pre-inserted with separators to distinguish between general clauses and terms. For particularly long contexts, an additional type of separator was pre-inserted to prevent the input from exceeding the length limit of the LLM.

Upon completion of the documentation, the indexing process is initiated. Initially, the proposed node parsing method is employed to obtain the node information. Subsequently, the nodes are written into the property graph, and the relationships between the nodes are connected. Ultimately, the text or caption property of the nodes is read, and the embedding model is used to embed these property values. The embedding vectors are then written into the vector database.

This example demonstrates the proposed RAG system from input to response. The demonstration draws upon the complex query and subqueries that have been provided in Table 4. The example generates three subqueries for the given query. It then proceeds to perform text embedding on the query and subqueries separately to obtain the embedding vectors. Initially, the top 3 nodes with the highest scores are retrieved for the query and subquery, respectively, based on cosine similarity. Subsequently, node post-processing is performed to obtain the correction scores of the nodes. Ultimately, the top 10 nodes with the highest correction scores are used to synthesize the response request. The retrieval and response results are shown in Table 6.

The proposed RAG system was able to successfully retrieve the 2 expected nodes with high scores and performed correct reasoning and answering.

Table 6
Retrieval and response results for the example.

Item	Retrieved nodes context	Lable	Score	Correction score
Query	Strength standard for ordinary steel bars (N/mm^2)	TableNode	0.54	0.54
	For structures with seismic fortification requirements, the performance of their longitudinal load-bearing steel bars should …	TextNode	0.49	0.50
	The reinforcement requirements for constructing edge components should …	TextNode	0.48	0.49
Subquery 1	Steel bars should be straightened using mechanical equipment or cold drawing methods …	TextNode	0.58	0.58 (expected)
	After determining the minimum lap length of longitudinal tensile steel bars according to …	TextNode	0.55	0.56
	The total elongation limit of ordinary steel bars and prestressed steel bars under maximum force should be …	TextNode	0.53	0.53
Subquery 2	The inner diameter of the bending arc of steel bars shall comply with the following regulations …	TextNode	0.54	0.55 (expected)
	The stirrups in the column shall comply with the following regulations …	TextNode	0.51	0.52
	The steel bars set in the horizontal mortar joints of the masonry should be …	TextNode	0.47	0.47 (deprecated)
Subquery 3	The ends of stirrups and tie bars should be hooked according to design requirements …	TextNode	0.51	0.52
	The length of the straight section of the longitudinally stressed steel bars after bending should comply with …	TextNode	0.48	0.49
	Concrete beams should use stirrups as reinforcement to withstand shear forces …	TextNode	0.47	0.47 (deprecated)
Response	"Yes, there are problems with the steel bar properties as specified. The cold-drawn rate should not exceed 1 %, and the bending diameter should be at least 80 mm for a 20 mm diameter rebar of 400 MPa strength."			

4.2. Task 1: basic queries

One of the most common applications of RAG is to answer user questions. This is a fundamental capability. We posed 120 questions and entered them as queries into the RAG framework. For example, "What should be the daily masonry height for masonry work?" These questions can be found in the original documents with clear answers. The subquery generation mechanism is temporarily disabled because queries for this task do not need to be decomposed. Since a single question may involve multiple expected nodes, we set k to 3, which is a value that is small enough and allows us to accommodate all expected nodes. Each question was manually labeled with the expected node IDs to evaluate the retrieval performance. The output answers were also manually checked to evaluate the response performance.

4.3. Task 2: risk and error recognition in complex queries

The recognition of risk and error in engineering behaviors and information has been a longstanding industry concern. To further evaluate the effectiveness of the proposed RAG in response to complex problems, we compiled 50 complex queries. To ensure consistency in the observations, each query randomly contained 3 correct propositions and 3 propositions containing potential errors or risks. These propositions are no longer explicit questions as in Task 1, but rather correct, incorrect, risky, or misleading statements. For example, "The minimum clear height of the basement should not be less than 1.8 m" The number of subqueries is strictly limited to a maximum of 3, which is the same as the number of problematic propositions. To ensure consistency in the number of returned nodes, the top 3 most similar nodes were retrieved for the original query and each subquery. The value of k was set to 10. As in Task 1, each query was manually labeled with the expected node IDs, and the final output answers were manually checked.

4.4. Task 3: open text generation

In certain scenarios, users may want the LLM to make a plan to handle engineering issues properly. This could include an acceptance plan for building materials, a seasonal construction plan, an environmental modification plan, and so on. These queries are characterized by high ambiguity and openness [57]. In this test, 25 queries at different stages of the building lifecycle are prepared to evaluate the open content generation capability of the proposed framework. As the queries for this task cannot be explicitly decomposed, the subquery generation mechanism is temporarily disabled. The value of k is still set to 10. The queries no longer have definitive answers. To conduct a fair evaluation, the three methods were ranked on five dimensions: (1) professionalism of sentence style, (2) correctness of conclusions, (3) completeness of important issues, (4) detail of elaboration, and (5) logic of the overall output. The average reciprocal was taken as the score. The output responses were sent anonymously to three senior experts in the field of buildings for ranking.

5. Results and discussion

5.1. Evaluation results of task 1

The evaluation results for Task 1 are shown in Table 7. Among the evaluation metrics, HR, recall and MAP are used for retrieval evaluation, while faithfulness, relevancy, and accuracy are used for response evaluation. As the native LLM does not entail the retrieval process, only the final response accuracy is evaluated in this case.

The most common application scenario for RAG is direct user questions about specific knowledge in the building lifecycle. It can be observed that the proposed method has resulted in notable advancements in both retrieval and response performance, with an accuracy of 94.17 % on the final response.

Due to the inherent complexity and the abstract nature of domain knowledge, the baseline RAG system is less likely to retrieve the expected nodes. Improved retrieval performance signifies that the RAG system is capable of more effectively eliminating distractions, aligning with user intent, and subsequently retrieving nodes with a higher degree of precision. Concurrently, the accuracy of retrieval results also introduces the potential for more reasonable responses. The most substantial improvements can be seen in the recall and

Table 7
Evaluation results for Task 1.

Method	HR (%)	Recall (%)	MAP (%)	Faithfulness (%)	Relevancy (%)	Accuracy (%)
Proposed framework	99.17	98.75	95.35	99.17	100.00	94.17
Baseline	77.50	68.61	56.27	83.33	80.83	69.17
Native LLM	–	–	–	–	–	43.33

HR: hit ratio, the proportion of retrieval results that contain at least one expected node.
Recall: the proportion of retrieved expected nodes to all expected nodes.
MAP: mean average precision, often considered as order-sensitive recall in top-k retrieval.
Faithfulness: whether the response is faithful to the retrieved nodes (i.e., whether there is a hallucination).
Relevancy: whether the retrieved nodes and responses are relevant and consistent with the query.
Accuracy: whether the output response is correct.

MAP, which indicate that enhanced node quality and an optimized retrieval strategy can markedly enhance retrieval performance. It also means that expected nodes are more likely to be retrieved with high rankings. Fixed length-based chunking is likely to destroy the complete semantics of a segment, which may lead to difficulties in the retrieval of at least one of the nodes. Additionally, in a few instances, similar queries and nodes fail to achieve high vector similarity. This is typically attributable to the fact that some important semantics inherent in the queries are exceptionally ignored, or that the embedding model is unable to fully comprehend human intent. The improved index structure and retrieval strategy can mitigate this phenomenon and increase the robustness of the proposed framework.

Response results are oriented directly to the user and answer the query, thus constituting the most intuitive aspect of evaluating the performance of a RAG system. The improved RAG system has made substantial progress on the response side, suggesting that it can provide more accurate and reasonable answers to explicit user questions, or, in other words, better fulfill the user's need to acquire specific knowledge. It is notable that the faithfulness and relevancy of LLM responses are influenced by retrieval performance. In instances where the LLM is unable to ascertain the answer to a query from the retrieved results, it will be more likely to hallucinate or discard the context to answer directly.

When the query involves unique regulations or cutting-edge issues, it is almost impossible for the native LLM to provide the correct answer. Nevertheless, an accuracy of 43.33 % is an outstanding achievement, particularly given that the LLM is not specifically fine-tuned for the domain of buildings. This is one of the key considerations in introducing RAG.

5.2. Evaluation results of task 2

Task 2 uses similar evaluation metrics as Task 1 does. The difference is that MAP is not used because the task does not have an explicit order sensitivity to the retrieved nodes. The evaluation results for Task 2 are shown in Table 8.

Simple knowledge quizzes, while common, clearly do not meet other possible query requirements. It can be found that the recall of both the proposed framework and the baseline decreased to varying degrees as the complexity of the task increased. Correspondingly, the accuracy also decreased to varying degrees. Nevertheless, the improved RAG system has demonstrated substantial advancement across all metrics.

The gap between the two RAG systems is further widened in terms of retrieval performance, especially recall. This indicates that the improved method can better find key information in complex problems and retrieve the corresponding nodes accurately. The subquery generation mechanism decomposes the complex query into a number of simple queries, which allows the recall to reach approximately 78 % in Task 1. Concurrently, the hit rate of the proposed framework can be considered unaffected by the complexity of the task. This indicates that each query can retrieve at least one expected node, and the response is more likely to include at least part of the correct answers. It is noteworthy that the number of subqueries typically represents a constraint on the retrieval performance during the retrieval stage. Relaxing the number restriction and the k value usually shows a modest increase in retrieval performance.

The gap between the two RAG systems also widens in terms of response performance. The improved RAG system is capable of producing more precise and reasonable responses to complex queries involving reasoning and multiple propositions. The proposed framework effectively resists the decline in faithfulness and relevancy, while the baseline approach shows a significant decline in both metrics compared to Task 1. This suggests that the improved RAG system is more controllable and traceable in its reasoning for complex problems and is less prone to hallucinations.

There are two noteworthy phenomena in terms of accuracy. First, the two RAG systems demonstrate higher accuracy than their recall, yet the accuracy of the baseline system is lower than that of the native LLM. This phenomenon can be attributed to the fact that when the relevance of the context to the query decreases, there are two tendencies in the response of the LLM. The first tendency is to rely on its native performance to make some degree of reasoning, which discards context but improves accuracy. The second tendency is to consider certain issues as unavailable for judgment due to insufficient information and to cease exercising its own reasoning power. This finding suggests that decreased retrieval performance may impede the native reasoning and response capabilities of LLM. Second, the native LLM demonstrated superior performance in Task 2 compared to Task 1. Subsequent analysis of the responses revealed that the native LLM is more effective at determining the reasonability of a proposition than at directly outputting a reasonable requirement. In other words, the native LLM is unable to guarantee that the details and boundaries of the answer will always align with the user's expectations, but it is more adept at identifying potential issues with a clause. This capability also contributes to the accuracy of a RAG system.

5.3. Evaluation results of task 3

The evaluation results for Task 3 are shown in Table 9. In addition to the five dimensions used to calculate the scores, we also provide faithfulness and relevancy as supplementary reference points.

Table 8
Evaluation results for Task 2.

Method	HR (%)	Recall (%)	Faithfulness (%)	Relevancy (%)	Accuracy (%)
Proposed framework	100.00	76.67	88.00	92.00	80.00
Baseline	76.00	28.43	32.00	34.00	34.00
Native LLM	–	–	–	–	56.00

Table 9
Evaluation results for Task 3.

Method	Professionalism	Correctness	Completeness	Details	Logic	Score	Faithfulness (%)	Relevancy (%)
Proposed framework	0.8733	0.8933	0.6533	0.8267	0.6333	0.7760	96.00	100.00
Baseline	0.5267	0.5867	0.6067	0.5533	0.4533	0.5453	48.00	88.00
Native LLM	0.4333	0.3533	0.5733	0.4267	0.7467	0.5067	–	–

The enhancement of content generation capabilities has the potential to provide users with more specific guidance for their ambiguous goals. It is important to note that the evaluation scores for Task 3 are inherently relative because they are derived from rankings. Even though there may be a large difference in the scores, all three methods achieved commendable performance. A low score does not necessarily indicate poor performance but rather a relative weakness compared to the other two methods in most cases.

Overall, the proposed framework offers a more significant improvement in response, while the baseline approach does not exhibit a notable improvement over the native LLM. In most cases, the proposed framework is more professional, correct, and detailed in its responses. These advances suggest that the improved RAG system can provide users with more knowledgeable and practical answers. The improvement in terms of the completeness of important issues is not considerable because the granularity of nodes in the hybrid index is much finer than that in the baseline approach. This occasionally results in relevant nodes being excluded when their quantity surpasses the k value. Conversely, a single text chunk in the baseline approach may cover multiple relevant semantic passages. Nevertheless, the improved system could still be more thoughtful in considering all aspects related to the query.

The baseline approach achieved mid-range performance on all the metrics. The lack of domain knowledge resulted in shortcomings in professionalism, correctness, and detail on the part of the native LLM. However, it performed better in terms of logic and achieved a slight advantage over the baseline approach in terms of the completeness of important issues. This can be attributed to the native LLM's tendency to take care of all aspects of a problem rather than focus on the core. This finding also indicates that including much domain knowledge in the context is not conducive to the logic and completeness of the LLM in dealing with problems.

6. Conclusions

In this study, we propose an improved RAG framework for the domain of buildings, including a hybrid index based on vectors, a property graph, and keywords, along with a corresponding hybrid retrieval strategy. The hybrid index contains five types of nodes and two types of relationships. The index employs a property graph to represent its structure and store knowledge, and a vector database to manage the embedding vectors. Moreover, a node parsing method is provided to support automated extraction and validation of node information. In terms of the hybrid retrieval strategy, a customized retrieval process is first designed according to the index structure. Furthermore, the query transformation and node post-processing modules are improved to achieve better retrieval and response performance in complex problems. To conduct a comprehensive evaluation of the proposed framework, we compiled original engineering documents containing knowledge about each stage of the building lifecycle and designed three evaluation tasks based on possible application scenarios. The evaluation results demonstrate that the performance of the proposed framework is significantly improved in each task compared to the baseline RAG and the native LLM. This study also suggests that the improved RAG can serve as an effective approach for knowledge management and utilization in the domain of buildings.

Despite the superior results achieved by the proposed framework in the tests, several constraints remain that affect the performance. One of the main aspects is the lack of further support for multimodal knowledge in the current stage of RAG. While current LLMs have demonstrated enhancements in their comprehension of simple tables, their capacity to respond to complex or usage-specific tables could be further refined. Meanwhile, the improved framework's processing of images is based on vectors of captions or relationships between nodes, rather than the computation of the images themselves. This precludes the images from being accurately comprehended by the RAG system, resulting in degradation in retrieval and response performance. Future research will focus more on the integration of RAG with large multimodal models and multimodal databases to better handle the representation, computation, similarity matching, and reasoning of multimodal data.

Another topic worthy of further investigation is the optimal approach for processing mathematical descriptions. Our findings indicate that when a node is primarily concerned with describing complex mathematical problems, its similarity to queries becomes unstable. In some cases, these descriptions are insufficiently similar to related queries, while in others, they are overly similar to unrelated queries. This phenomenon can be attributed primarily to the abstract nature of these nodes in their presentation and the absence of textual generalization. Furthermore, the mathematical ability of LLMs decreases rapidly as the complexity of the problems increases. This suggests that incorrect computation or reasoning may still occur despite successful retrieval of the expected nodes. In regard to the retrieval process, one possible enhancement is the provision of concise summaries for mathematical descriptions, which has the potential to improve retrieval stability. On the response side, the implementation of specialized advanced reasoning models or external tool functions is a promising research direction for addressing these limitations.

CRediT authorship contribution statement

Zhiqi Wang: Writing – original draft, Visualization, Validation, Methodology, Conceptualization. **Zhongcun Liu:** Supervision, Project administration, Investigation, Funding acquisition. **Weizhen Lu:** Validation, Investigation, Funding acquisition, Data curation. **Lu Jia:** Writing – review & editing, Supervision, Resources, Funding acquisition.

Data availability

Some or all data, models, or code that support the findings of this study are available from the corresponding author upon reasonable request.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used DeepL, Grammarly, and Curie in order to improve readability and language of the work. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

Funding

This work was supported by the Science and Technology Plan Project of Jiangxi Geological Bureau (Grant No. 2023JXDZKJKY07); the Science and Technology Plan Project of Jiangxi Geological Bureau (Grant No. 2021JXDZ70001); and the Science and Technology Plan Project of Jiangxi Coalfield Geology Bureau (Grant No. 2020JXMD70003).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] J. WątróBSKI, Towards knowledge handling in sustainable management domain, Procedia Comput. Sci. 159 (2019) 1591–1601.
[2] T. Koç, K. Kurt, A. AkbıYıK, A brief summary of knowledge management domain: 10-Year history of the journal of knowledge management, Procedia Comput. Sci. 158 (2019) 891–898.
[3] A.A. Markus, B.W. Hobson, H.B. Gunay, S. Bucking, Does a knowledge gap contribute to the performance gap? Interviews with building operators to identify how data-driven insights are interpreted, Energy Build. 268 (2022) 112238.
[4] G. Schreiber, Knowledge engineering, Foundations of Artificial Intelligence 3 (2008) 929–946.
[5] W. J.C. Verhagen, P. Bermell-Garcia, R. E.C. Van Dijk, R. Curran, A critical review of Knowledge-Based Engineering: an identification of research challenges, Adv. Eng. Inform. 26 (1) (2012) 5–15.
[6] B. Zhong, L. Ding, P.E. Love, H. Luo, An ontological approach for technical plan definition and verification in construction, Autom. ConStruct. 55 (2015) 47–57.
[7] H. Paulheim, Knowledge graph refinement: a survey of approaches and evaluation methods, Semant. Web 8 (3) (2017) 489–508.
[8] J. Chen, A. Agarwal, S. Abdelkarim, et al., Reltransformer: a transformer-based long-tail visual relationship recognition, in: Proceedings of the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, F, 2022 [C].
[9] J. Huang, K.C.-C. Chang, Towards reasoning in large language models: a survey, arXiv preprint arXiv:2212.10403, (2022).
[10] J. Wei, X. Wang, D. Schuurmans, et al., Chain-of-thought prompting elicits reasoning in large language models, Adv. Neural Inf. Process. Syst. 35 (2022) 24824–24837.
[11] D. Zhou, N. SchäRLI, L. Hou, et al., Least-to-most prompting enables complex reasoning in large language models, arXiv preprint arXiv (2022) 2205.10625.
[12] Y. Chang, X. Wang, J. Wang, et al., A survey on evaluation of large language models, ACM Transactions on Intelligent Systems and Technology 15 (3) (2024) 1–45.
[13] M.O. Topal, A. Bas, I. Van Heerden, Exploring transformers in natural language generation: gpt, bert, and xlnet, arXiv preprint arXiv:2102.08036, (2021).
[14] X. Wang, Y. Yan, L. Huang, et al., Hallucination detection for generative large language models by bayesian sequential estimation, Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, F, (2023)15361-15371.
[15] Z. Ji, N. Lee, R. Frieske, et al., Survey of hallucination in natural language generation, ACM Comput. Surv. 55 (12) (2023) 1–38.
[16] X. Wang, Z. Wang, X. Gao, et al., Searching for best practices in retrieval-augmented generation, Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (2024) 17716–17736.
[17] C.J. Anumba, C. Egbu, P. Carrillo, Knowledge Management in Construction [M], John Wiley & Sons, 2008.
[18] J.M. Kamara, G. Augenbroe, C.J. Anumba, P.M. Carrillo, Knowledge management in the architecture, engineering and construction industry, Constr. Innov. 2 (1) (2002) 53–67.
[19] L. Kanapeckiene, A. Kaklauskas, E.K. Zavadskas, M. Seniut, Integrated knowledge management model and system for construction projects, Eng. Appl. Artif. Intell. 23 (7) (2010) 1200–1215.
[20] D. Yu, J. Yang, Knowledge management research in the construction industry: a review, J. Knowl. Econ. 9 (2018) 782–803.
[21] W. Jiang, Y. Liu, K. Chen, et al., Early-warning of unsafe hoisting operations: an integration of digital twin and knowledge graph, Develop. Built Environ. (2024) 100490.
[22] M.H. Rasmussen, M. LefrançOIS, P. Pauwels, et al., Managing interrelated project information in AEC Knowledge Graphs, Autom. ConStruct. 108 (2019) 102956.
[23] W. Wu, C. Wen, Q. Yuan, et al., Construction and application of knowledge graph for construction accidents based on deep learning, Eng. Construct. Architect. Manag. 32 (2) (2025) 1097–1121.
[24] Z. Pan, C. Su, Y. Deng, J. Cheng, Video2Entities: a computer vision-based entity extraction framework for updating the architecture, engineering and construction industry knowledge graphs, Autom. ConStruct. 125 (2021) 103617.
[25] J. Achiam, S. Adler, S. Agarwal, et al., Gpt-4 technical report, arXiv preprint arXiv (2023) 2303.08774.
[26] P. Lewis, E. Perez, A. Piktus, et al., Retrieval-augmented generation for knowledge-intensive nlp tasks, Adv. Neural Inf. Process. Syst. 33 (2020) 9459–9474.
[27] Y. Gao, Y. Xiong, X. Gao, et al., Retrieval-augmented generation for large language models: a survey, arXiv preprint arXiv (2023) 2312.10997.
[28] S. Wu, Y. Xiong, Y. Cui, et al., Retrieval-augmented generation for natural language processing: a survey, arXiv preprint arXiv (2024) 2407.13193.
[29] H. Li, Y. Su, D. Cai, et al., A survey on retrieval-augmented text generation, arXiv preprint arXiv (2022) 2202.01110.
[30] S. Zeng, J. Zhang, P. He, et al., The good and the bad: exploring privacy issues in retrieval-augmented generation (rag), arXiv preprint arXiv:2402.16893 (2024).
[31] P. Zhao, H. Zhang, Q. Yu, et al., Retrieval-augmented generation for ai-generated content: a survey, arXiv preprint arXiv (2024) 2402.19473.
[32] T. Mikolov, Efficient estimation of word representations in vector space, arXiv preprint arXiv (2013) 1301.3781.
[33] J. Devlin, M.W. Chang, K. Lee, et al., Bert: pre-training of deep bidirectional transformers for language understanding, Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, Volume 1 (long and short papers) (2019) 4171-4186.

[34] V. Karpukhin, B. Oğuz, S. Min, et al., Dense passage retrieval for open-domain question answering, EMNLP 1 (2020) 6769–6781.
[35] B. Peng, Y. Zhu, Y. Liu, et al., Graph retrieval-augmented generation: a survey, arXiv preprint arXiv (2024) 2408.08921.
[36] X. Ma, Y. Gong, P. He, et al., Query rewriting for retrieval-augmented large language models, Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (2023) 5303–5315.
[37] C.-M. Chan, C. Xu, R. Yuan, et al., Rq-rag: learning to refine queries for retrieval augmented generation, arXiv preprint arXiv:2404.00610, (2024).
[38] L. Gao, X. Ma, J. Lin, J. Callan, Precise zero-shot dense retrieval without relevance labels, Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, Volume 1: Long Papers (2023) 1762-1777.
[39] L. Wang, N. Yang, F. Wei, Query2doc: query expansion with large language models, arXiv preprint arXiv (2023) 2303.07678.
[40] J. Johnson, M. Douze, H. JéGOU, Billion-scale similarity search with GPUs, IEEE Transactions on Big Data 7 (3) (2019) 535–547.
[41] E. Song, S. Kim, H. Lee, et al., Re3val: reinforced and reranked generative retrieval, arXiv preprint arXiv:2401.16979 (2024).
[42] S. Zhuang, G. Zuccon, TILDE: term independent likelihood moDEl for passage re-ranking, in: Proceedings of the Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, F, 2021 [C].
[43] S. Zhuang, G. Zuccon, Fast passage re-ranking with contextualized exact term matching and efficient passage expansion, arXiv preprint arXiv (2021) 2108.08513.
[44] N.F. Liu, K. Lin, J. Hewitt, et al., Lost in the middle: how language models use long contexts, Transactions of the Association for Computational Linguistics 12 (2024) 157–173.
[45] O. Ram, Y. Levine, I. Dalmedigos, et al., In-context Retrieval-Augmented Language Models, vol. 11, Transactions of the Association for Computational Linguistics, 2023, pp. 1316–1331.
[46] Y. Wu, M.N. Rabe, D. Hutchins, C. Szegedy, Memorizing transformers, arXiv preprint arXiv (2022) 2203.08913.
[47] C. Zelin, W.K. Chung, M. Jeanne, et al., Rare disease diagnosis using knowledge guided retrieval augmentation for ChatGPT, J. Biomed. Inf. 157 (2024) 104702.
[48] M. Quidwai, S. Thibaud, J. Richter, et al., P-145 innovative AI-driven decision support tool for multiple myeloma using retrieval augmented generation, Clin. Lymphoma Myeloma Leuk. 24 (2024) S123–S124.
[49] C. Ye, Exploring a learning-to-rank approach to enhance the Retrieval Augmented Generation (RAG)-based electronic medical records search engines, Informatics and Health 1 (2) (2024) 93–99.
[50] M. Alkhalaf, P. Yu, M. Yin, C. Deng, Applying generative AI with retrieval augmented generation to summarize and extract key clinical information from electronic health records, J. Biomed. Inf. (2024) 104662.
[51] T. Feridooni, A.P. Javidan, D.N. Mahmood, et al., Development of a Vascular Surgery–specific Artificial Intelligence Chat Interface Using Retrieval-Augmented Generation: VASC. AI, A Specialized Vascular Surgery Chatbot [J], JVS-Vascular Insights, 2024 100137.
[52] D.-H. Yao, A. Badhwar, W. Li, et al., 134 emergency department admitting service triage using retrieval-augmented language models, Ann. Emerg. Med. 84 (4) (2024) S63.
[53] A. Chandrasekhar, J. Chan, F. Ogoke, et al., AMGPT: a large language model for contextual querying in additive manufacturing, Add. Manuf. Lett. 11 (2024) 100232.
[54] L. Siddharth, J. Luo, Retrieval augmented generation using engineering design knowledge, Knowl. Base Syst. 303 (2024) 112410.
[55] S. Wong, C. Zheng, X. Su, Y. Tang, Construction contract risk identification based on knowledge-augmented language models, Comput. Ind. 157 (2024) 104082.
[56] J. Li, A. Sun, J. Han, C. Li, A survey on deep learning for named entity recognition, IEEE Trans. Knowl. Data Eng. 34 (1) (2020) 50–70.
[57] B. Jiang, J. Yang, C. Yang, et al., Knowledge augmented dialogue generation with divergent facts selection, Knowl. Base Syst. 210 (2020) 106479.